

Projet — Le robot de réconfort

Master informatique — projet de semestre, par binôme

SOMMAIRE

1. Contexte
2. Ce que vous rendez
- 2 bis. La vidéo de démonstration
3. Le monde
4. Le déroulement
5. Les fichiers
6. Les règles de décision
7. Ce que vous devez construire
8. Tests
9. La base de code fournie
10. Comment vous êtes évalués
11. Le rapport (3 pages maximum)
12. Projet 2 — plusieurs robots
- Annexe A — La roue de Plutchik
- Annexe B — Se déplacer dans une grille
- Annexe C — Naviguer dans l'armoire
- Annexe D — Une simulation complète

1. Contexte

Une résidence est équipée d'un robot de service. Les résidents lui envoient des messages courts depuis leur chambre. Pour chaque message, le robot doit aller consulter un dictionnaire posé près de l'armoire pour savoir quelle émotion le message exprime, fouiller l'armoire pour y trouver l'objet de réconfort correspondant, puis le rapporter au résident.

Le robot ne connaît pas le plan des lieux. Il connaît les dimensions de l'appartement, sa propre position, celle de l'armoire, celle du dictionnaire et celle de chaque résident, mais **pas les murs**. Il les découvre en se déplaçant, comme un aspirateur autonome le ferait.

Vous écrivez ce robot.

Le projet se déroule en deux temps :

	SUJET	RENDU
Projet 1	un seul robot (sections 2 à 11)	fin de semestre
Projet 2	plusieurs robots qui se coordonnent (section 12)	second rendu

Le projet 2 réutilise directement le code du projet 1. Une architecture propre au premier rendu vous fera gagner beaucoup de temps au second.

2. Ce que vous rendez

Une archive `NOM1_NOM2.zip` contenant, à sa racine :

1. **le code source** de votre robot ;
2. **un fichier `rendu.json`** décrivant comment lancer votre programme (voir plus bas) ;
3. **vos tests**, et la commande qui les lance, indiquée dans le README ;
4. **un README** : comment lancer, comment tester, ce qui marche, ce qui ne marche pas. Un squelette à recopier vous est fourni dans `base_fournie/MODELE_README.md` ;
5. **un rapport** de 3 pages maximum (section 11) ;
6. **une vidéo de démonstration** de 1 min 30 maximum (section 2 bis) ;
7. **le dépôt Git** de votre travail (dossier `.git` inclus, ou une URL).

Le contrat d'appel

Votre programme est lancé avec **quatre arguments**, dans cet ordre :

```
<chemin de la carte> <chemin du scénario> <dossier de données> <chemin de la trace à écrire>
```

Le dossier de données contient `dictionnaire.json` et un ou plusieurs fichiers d'armoire ; **le nom de l'armoire à utiliser est donné par le scénario**, pas par le nom du dossier. Ne codez en dur aucun chemin.

Vous déclarez la commande de lancement dans `rendu.json` :

```
{
  "format": "robot-reconfort/rendu",
  "version": 1,
  "equipe": ["DUPONT Marie", "MARTIN Paul"],
  "langage": "Python 3",
  "preparation": [],
  "commande": ["python", "agent.py"],
  "video": "demo.mp4"
}
```

À la correction, les quatre arguments sont ajoutés à la fin de `commande`. Si votre langage demande une compilation, mettez-la dans `preparation`, qui est une liste de commandes exécutées une seule fois avant tout le reste — par exemple `[["javac", "-d", "classes", "src/Agent.java"]]` avec `"commande": ["java", "-cp", "classes", "Agent"]`.

Votre programme doit rendre le code de retour **0** en cas de succès et un code **non nul** en cas d'erreur de données (section 5.6).

Langage

Python est conseillé et une base de code vous est fournie en Python (section 9). Vous êtes libres d'utiliser un autre langage : dans ce cas, c'est à vous de réimplémenter la lecture des fichiers et l'écriture de la trace en respectant exactement les formats de la section 5. Le contrat d'appel, lui, est le même pour tout le monde — c'est justement à ça que sert `rendu.json`.

Rien n'exige un système particulier : tout se fait avec quatre arguments de ligne de commande, qui s'écrivent pareil sous Windows, macOS et Linux.

Le champ `video` donne le nom du fichier vidéo joint à l'archive, ou une URL si vous l'hébergez ailleurs (dans ce cas, vérifiez que le lien est accessible sans compte).

2 bis. La vidéo de démonstration

1 min 30 maximum, capture d'écran. **Pas de voix off, pas de montage, pas d'introduction** : on doit simplement voir votre robot en marche sur une carte, du départ à la livraison.

Un affichage texte dans le terminal suffit largement : la grille redessinée à chaque pas, avec la position du robot. Rien n'impose une interface graphique, et écrire cet affichage vous servira de toute façon à déboguer.

Si votre affichage le permet, cadrez un moment où le robot se cogne à un mur et change d'itinéraire. C'est ce qui rend une démonstration convaincante en quelques secondes.

Format libre — mp4, mkv, gif animé — joint à l'archive, ou lien accessible sans compte. Dans les deux cas, déclarez-le dans le champ `video` de votre `rendu.json`.

3. Le monde

3.1 L'appartement

L'appartement est une grille rectangulaire. Chaque case porte l'un des symboles suivants :

SYMBOLE	SIGNIFICATION	TRAVERSABLE
#	mur	non
.	case libre	oui
R	case de départ du robot (libre elle aussi)	oui
A	armoire	non
D	dictionnaire	non
P	résident	non

Les coordonnées sont des couples **[ligne, colonne]**, indexés à partir de 0, l'origine étant le coin **haut-gauche**. Les quatre directions sont :

DIRECTION	EFFET
N	ligne - 1
S	ligne + 1
E	colonne + 1
O	colonne - 1

Le bord de la grille est toujours constitué de murs, et il n'y a pas de déplacement en diagonale.

L'armoire, le dictionnaire et les résidents occupent une case **non traversable**. Pour interagir avec l'un d'eux, le robot doit se placer sur une case libre **orthogonalement adjacente**.

3.2 La perception est locale

C'est la **règle centrale du sujet**. Au début de la simulation, votre robot connaît uniquement :

- les dimensions de la grille ;
- sa position de départ ;
- la position de l'armoire et celle du dictionnaire ;
- l'identifiant, le nom et la position de chaque résident.

Il ne connaît **aucun mur**. À chaque tour, et seulement à ce moment-là, il perçoit le type des quatre cases orthogonalement adjacentes à sa position courante. Rien de plus : pas de vision en ligne droite, pas de portée de deux cases, pas de diagonale.

Interdit : lire le champ `grille` du fichier de carte depuis votre agent, sous quelque forme que ce soit. Ce champ existe pour le simulateur, qui répond aux perceptions ; le robot, lui, n'y a pas accès. Cette règle est vérifiée automatiquement à la correction, et l'enfreindre annule les points de la partie fonctionnelle.

Votre robot doit donc maintenir sa propre **carte mentale**, la mettre à jour à chaque perception, planifier sur cette carte incomplète, et **replanifier** quand la réalité contredit son plan. L'annexe B explique comment, si le sujet vous est nouveau.

3.3 L'armoire est une seconde grille

L'armoire est un meuble à casiers : **8 colonnes**, une par émotion, et **3 lignes**, une par intensité.

JOIE	CONFIANCE	PEUR	SURPRISE	TRISTESSE	DÉGOÛT	COLÈRE	ANTICIPATION
faible							
moyenne							
forte							

Le casier [ligne, colonne] est donc repéré par ligne = l'indice de l'intensité (0 faible, 1 moyenne, 2 forte) et colonne = l'indice de l'émotion dans l'ordre ci-dessus.

Le robot ne prend pas un objet à distance : il déplace un **sélecteur** d'un casier à la fois, avec l'action **CHERCHER**. Les colonnes **bouclent** — la colonne 7 est voisine de la colonne 0, parce que l'armoire est rangée selon la roue de Plutchik (annexe A) — mais les lignes ne bouclent pas.

L'itinéraire du sélecteur est libre. Pour aller d'un casier à un autre, vous pouvez bouger d'abord verticalement puis horizontalement, ou l'inverse, ou alterner : les trois sont acceptés, et ils coûtent le même nombre de mouvements. Ils ne font simplement pas passer devant les mêmes casiers en chemin — ce qui, comme on va le voir, n'est pas sans conséquence. L'annexe C détaille tout cela.

Et surtout : le robot **ne sait pas ce qu'il y a dans un casier tant qu'il ne s'est pas placé devant**. Certains casiers sont vides. C'est en fouillant qu'il l'apprend.

Interdit, pour la même raison qu'en 3.2 : lire le champ **objet** des casiers dans le fichier d'armoire. Vous avez le droit de lire la disposition (ligne, colonne, émotion, intensité), qui est écrite sur les étiquettes du meuble ; le contenu, lui, se découvre en regardant. C'est vérifié à la correction : annoncer qu'un casier était vide sans être allé le voir est une erreur.

3.4 Les actions

À chaque tour, le robot exécute exactement une action.

ACTION	ARGUMENT	CONDITIONS
AVANCER	N, S, E ou 0	la case visée est libre
CONSULTER	aucun	le robot est adjacent au dictionnaire
CHERCHER	N, S, E ou 0	le robot est adjacent à l'armoire ; déplace le sélecteur d'un casier
PRENDRE	le nom de l'objet	le robot est adjacent à l'armoire, le casier courant n'est pas vide, et le robot ne transporte rien
DONNER	le nom de l'objet	le robot est adjacent au résident concerné et transporte cet objet
ATTENDRE	aucun	toujours

Le robot transporte **au plus un objet à la fois**.

4. Le déroulement

Le scénario est une liste ordonnée de demandes. Pour chacune, dans l'ordre :

1. le robot reçoit le message et l'identifiant du résident ;

- il rejoint le dictionnaire et exécute **CONSULTER** : il apprend alors l'émotion et l'intensité du message (section 6.1) ;
- il rejoint l'armoire et la fouille jusqu'à trouver un casier acceptable et garni (section 6.2), puis exécute **PRENDRE** ;
- il rejoint le résident et exécute **DONNER** ;
- la demande suivante commence, depuis la position où il se trouve.

L'état persiste d'une demande à l'autre : la position du robot, sa carte mentale, et la position du sélecteur dans l'armoire.

5. Les fichiers

Tous les fichiers sont en **JSON encodé en UTF-8**. Tous portent un champ **format** et un champ **version** (valeur **1**), à vérifier avant toute chose.

5.1 **carte.json** — **robot-reconfort/carte**

```
{
  "format": "robot-reconfort/carte",
  "version": 1,
  "nom": "appartement_01",
  "dimensions": { "hauteur": 9, "largeur": 13 },
  "legende": { "#": "mur", ".": "libre", "R": "depart du robot",
    "A": "armoire", "D": "dictionnaire", "P": "resident" },
  "grille": ["#####", "#.....#....#", "..."],
  "depart_robot": [6, 1],
  "armoire": { "position": [6, 4] },
  "dictionnaire": { "position": [7, 6] },
  "residents": [
    { "id": "R1", "nom": "Elodie", "position": [6, 11] },
    { "id": "R2", "nom": "Daniel", "position": [3, 3] }
  ]
}
```

Rappel : **grille** est réservé au simulateur (section 3.2).

5.2 **dictionnaire.json** — **robot-reconfort/dictionnaire**

```
{
  "format": "robot-reconfort/dictionnaire",
  "version": 1,
  "nom": "dictionnaire_residence",
  "emotions": ["joie", "confiance", "peur", "surprise",
    "tristesse", "degout", "colere", "anticipation"],
  "intensites": ["faible", "moyenne", "forte"],
  "entrees": [
    { "formes": ["triste", "tristesse"],
      "emotion": "tristesse", "intensite": "moyenne" }
  ]
}
```

Les formes sont déjà en minuscules et sans accents, et une forme donnée n'apparaît que dans une seule entrée. L'ordre des émotions dans le champ `emotions` est l'ordre de la roue (section 6.2 et annexe A) : ne le réordonnez pas.

5.3 `armoire.json` — `robot-reconfort/armoire`

```
{
  "format": "robot-reconfort/armoire",
  "version": 1,
  "nom": "armoire_standard",
  "emotions": ["joie", "confiance", "..."],
  "intensites": ["faible", "moyenne", "forte"],
  "casier_depart": [1, 0],
  "casiers": [
    { "ligne": 1, "colonne": 4, "emotion": "tristesse",
      "intensite": "moyenne", "objet": "couverture" },
    { "ligne": 1, "colonne": 3, "emotion": "surprise",
      "intensite": "moyenne", "objet": null }
  ]
}
```

Un `objet` à `null` marque un casier vide. `casier_depart` donne la position du sélecteur au tout début de la simulation.

5.4 `scenario.json` — `robot-reconfort/scenario`

```
{
  "format": "robot-reconfort/scenario",
  "version": 1,
  "nom": "scenario_01",
  "carte": "appartement_01",
  "armoire": "armoire_standard",
  "demandes": [
    { "numero": 1, "resident": "R1",
      "message": "Je me sens tellement seule ce soir, personne n'est passé." }
  ]
}
```

Le champ `armoire` donne le **nom de fichier sans extension** à charger depuis le dossier de données.

5.5 `trace.json` — `robot-reconfort/trace`

C'est votre sortie, et c'est sur elle que vous êtes évalués. Elle contient **un pas par action** et **une livraison par demande**, réussie ou non.

```
{
  "format": "robot-reconfort/trace",
  "version": 1,
  "carte": "appartement_01",
  "scenario": "scenario_01",
  "equipe": ["DUPONT Marie", "MARTIN Paul"],
  "pas": [
    { "t": 0, "demande": 1, "position": [6, 1], "casier": [1, 0],
      "action": "AVANCER", "argument": "E",
      "perception": { "N": "libre", "S": "libre", "E": "libre", "O": "mur" },
      "contenu_casier": null, "commentaire": null }
  ],
  "livraisons": [
    { "demande": 1, "resident": "R1",
      "emotion": "tristesse", "intensite": "moyenne",
      "casier_choisi": [1, 4], "repli": "aucun",
      "objet": "couverture", "pas_utilises": 34,
      "succes": true, "motif_echec": null }
  ],
  "resume": { "demandes": 4, "reussies": 4, "echecs": 0, "pas_total": 160 }
}
```

Précisions :

- `t` est numéroté à partir de 0 sur toute la simulation, pas par demande ;
- `position` et `casier` sont ceux **avant** l'action ;
- `perception` est ce que le robot voit depuis cette position ; les valeurs sont `mur`, `libre`, `armoire`, `dictionnaire`, `resident`. Ce champ est vérifié à la correction : une perception qui ne correspond pas à la réalité est une erreur, même si le déplacement est correct ;
- `contenu_casier` est l'objet présent dans le casier courant lorsque le robot est devant l'armoire, et `null` sinon. Vérifié également ;
- `repli` vaut `aucun`, `intensite`, `voisine_1` ou `voisine_2` (section 6.2) ;
- en cas d'échec, `succes` est `false` et `motif_echec` contient une chaîne courte : `resident inaccessible`, `armoire vide` ou `emotion indeterminée`.

5.6 Ce que votre programme doit valider

Un fichier d'entrée peut être absent, mal formé ou incohérent. Votre programme doit alors **afficher un message d'erreur clair sur la sortie d'erreur et se terminer avec un code de retour non nul** — pas produire une trace d'exception, pas partir en boucle infinie, pas écrire une trace vide.

Sont à détecter, au minimum :

- fichier absent, non lisible, ou non encodé en UTF-8 ;
- JSON syntaxiquement invalide ;
- `format` ou `version` inattendus ;
- champ obligatoire absent, ou du mauvais type ;
- grille non rectangulaire, ou dimensions annoncées incohérentes ;
- position hors de la grille ;

- résident, armoire, dictionnaire ou départ du robot dont la position ne correspond pas au symbole attendu dans la grille ;
- deux résidents avec le même identifiant ;
- scénario prévu pour une autre carte, ou demande adressée à un résident absent de la carte ;
- casier hors des dimensions de l'armoire, casier en double, ou casier dont l'étiquette `emotion` / `intensite` ne correspond pas à sa position ;
- `casier_depart` hors de l'armoire ;
- émotion ou intensité inconnue, ou même mot dans deux entrées du dictionnaire.

Cette liste est notée. **Vingt-deux** fichiers volontairement cassés font partie du jeu de test caché. Pour chacun, la correction vérifie trois choses : code de retour non nul, message compréhensible sur la sortie d'erreur, aucun fichier de trace écrit.

6. Les règles de décision

Ces règles sont **imposées et déterministes**. Deux binômes corrects produisent les mêmes livraisons ; seuls les trajets peuvent différer.

6.1 Lire l'émotion dans le dictionnaire

Une fois le robot devant le dictionnaire :

1. passer le message en minuscules, retirer les accents, le découper en mots sur tout ce qui n'est pas une lettre ;
2. parcourir ces mots **de gauche à droite** ;
3. **le premier mot présent dans le dictionnaire donne la réponse** : son émotion et son intensité. On s'arrête là ;
4. si aucun mot n'est reconnu, la demande échoue avec le motif `emotion indeterminee`.

Exemple. « Je suis sidéré, la télévision s'est allumée toute seule. » donne les mots `je`, `suis`, `sidere`, `la`, `television`, `s`, `est`, `allumee`, `toute`, `seule`. Le premier reconnu est `sidere` → **surprise / forte**. Le mot `seule`, plus loin, est aussi dans le dictionnaire, mais il arrive après : il ne compte pas.

C'est tout. Pas de score, pas de moyenne, pas de comptage : une boucle et un `return`.

6.2 Choisir le casier

Les huit émotions sont disposées sur la **roue de Plutchik**, dans l'ordre du champ `emotions` :

```

              joie
    anticipation      confiance
colere             peur
    degout          surprise
              tristesse

```

La distance entre deux émotions est leur distance sur ce cycle : 0 pour elle-même, 1 pour une voisine, jusqu'à 4 pour l'émotion diamétralement opposée (`joie` ↔ `tristesse`, `confiance` ↔ `dégoût`, `peur` ↔ `colère`, `surprise` ↔ `anticipation`).

Soit (e, i) l'émotion et l'intensité lues dans le dictionnaire. On considère **toutes** les cases (e', i') dont la distance sur la roue $d(e, e')$ est **inférieure ou égale à 2**, et on les trie par la clé suivante, dans l'ordre :

1. $d(e, e')$ croissante ;
2. l'écart d'intensité $|\text{rang}(i) - \text{rang}(i')|$ croissant ;
3. l'intensité la plus faible d'abord ;
4. le sens horaire avant le sens antihoraire.

Le robot visite les casiers de cette liste **dans cet ordre**, et prend l'objet du premier qu'il trouve garni. Si les quinze casiers de la liste sont vides, la demande échoue avec le motif `armoire vide`.

Les émotions à distance 3 et 4 ne sont **jamais** utilisées : on ne répond pas à un chagrin par un objet de joie.

L'étiquette `repli` de la trace récapitule le résultat : `aucun` si le casier exact était garni, `intensite` si on est resté sur la même émotion, `voisine_1` ou `voisine_2` selon la distance sur la roue.

Rappel de 3.3 : la liste de candidats se calcule, mais leur contenu se **constate**. Vous ne pouvez annoncer un repli que si le robot est effectivement passé devant chacun des casiers plus prioritaires.

Trois précisions sur ce que « passer devant » veut dire :

- un casier **simplement traversé** compte comme vu : le sélecteur s'y est trouvé, la trace journalise son `contenu_casier`, le robot a donc bien constaté qu'il était vide ;
- l'**itinéraire n'est pas imposé** (section 3.3). Balayer toute l'armoire avant de choisir est légal, juste coûteux ; une route qui passe par les casiers prioritaires en allant vers sa cible est plus habile ;
- passer devant un casier **garni** ne dispense pas de suivre l'ordre. Si le robot croise `peur/forte` en chemin alors que `tristesse/forte` est plus prioritaire, c'est `tristesse/forte` qu'il doit prendre.

6.3 Les échecs

Un échec n'est pas un plantage. Le robot journalise la livraison en échec avec son motif, et **passé à la demande suivante**. Une demande impossible ne doit jamais interrompre le scénario.

Le cas le plus délicat est le résident inaccessible. Avec une perception locale, votre robot ne peut pas le savoir d'avance : il doit explorer jusqu'à avoir la **preuve** qu'aucun chemin n'existe. L'annexe B explique en quoi consiste cette preuve.

7. Ce que vous devez construire

Votre robot est un **agent**, pas un solveur de labyrinthe. Il doit faire apparaître, séparément et lisiblement dans votre code :

- **la perception** : réception des quatre cases adjacentes, et du contenu du casier courant quand il est devant l'armoire ;
- **la mémoire** : la carte mentale et ce qu'il a déjà vu dans l'armoire, mises à jour à chaque tour ;
- **la décision** : le choix de la prochaine action ;
- **l'action** : son exécution et sa journalisation.

Une fonction unique de 300 lignes qui fait tout est un échec de conception, même si elle produit la bonne trace.

Structures de données attendues

Vous aurez besoin, au minimum : d'un parcours de graphe pour la planification de chemin, d'une file pour ce parcours, d'une représentation de la carte mentale distincte de la carte réelle, et d'une structure cyclique pour la roue et les colonnes de l'armoire.

Ces structures sont à écrire. Vous n'avez pas le droit d'appeler une bibliothèque de plus court chemin (`networkx`, `scipy.sparse.csgraph`, `igraph`, ou l'équivalent dans votre langage). Les bibliothèques standard pour le JSON, les tests et l'affichage sont autorisées sans restriction.

8. Tests

Vous devez livrer des tests automatisés qui couvrent au moins :

1. **la planification** : sur une petite grille écrite à la main, un chemin dont vous connaissez la longueur ;
2. **la replanification** : un mur découvert en route change le trajet ;
3. **la lecture du dictionnaire** : un message par émotion, plus un message où deux mots connus apparaissent et où c'est bien le premier qui gagne, plus un message sans aucun mot reconnu ;
4. **le déplacement du sélecteur** dans l'armoire, y compris le bouclage entre la colonne 7 et la colonne 0 ;
5. **la règle de repli** : casier exact garni, casier vide avec repli en intensité, colonne entière vide avec repli sur une voisine, aucun casier garni à distance ≤ 2 ;
6. **la validation des fichiers** : un fichier cassé par catégorie de la section 5.6 ;
7. **le cas inaccessible** : un résident muré, sur une grille minuscule écrite à la main.

Un test qui ne peut pas échouer ne compte pas. Un test généré et jamais relu se voit.

9. La base de code fournie

Le dossier `base_fournie/` contient :

<code>reconfort_io.py</code>	lecture des 4 fichiers, écriture de la trace
<code>demo.py</code>	exemple d'utilisation, ne résout rien
<code>rendu.json</code>	modèle à compléter
<code>MODELE_README.md</code>	squelette du README de votre dépôt
<code>donnees/dictionnaire.json</code>	
<code>donnees/armoire_standard.json</code>	
<code>cartes/appartement_01..05.json</code>	
<code>cartes/scenario_01..05.json</code>	

`reconfort_io.py` fait **deux choses** : lire les entrées et écrire la trace. Il ne contient ni planification, ni fouille, ni règle de repli, ni boucle de décision — c'est votre travail. Ses validations sont volontairement minimales : il vérifie l'en-tête `format / version` et rien d'autre. Tout ce qui est listé en 5.6 reste à votre charge.

Vous pouvez le modifier. Vous devez le comprendre : les questions de soutenance porteront aussi sur ce fichier.

Pour démarrer :

```
cd base_fournie
python demo.py cartes/appartement_01.json cartes/scenario_01.json donnees sortie.json
```

10. Comment vous êtes évalués

Le jeu de test caché

Les cinq cartes et scénarios fournis servent à travailler. **La correction utilise en plus des jeux que vous n'aurez pas vus**, exécutés via votre `rendu.json`. Ils comportent notamment un appartement plus grand, une armoire dont une colonne entière est vide, un résident inaccessible, et les vingt-deux fichiers cassés de la section 5.6.

Un robot qui ne marche que sur les cartes fournies échouera. Testez sur des cartes que vous fabriquez vous-mêmes : c'est un exercice utile en soi.

La trace est rejouée automatiquement contre la réalité. Chaque déplacement, chaque perception annoncée, chaque casier ouvert, chaque objet pris et chaque échec sont vérifiés.

Barème du projet 1 (sur 20)

	POINTS
Cartes fournies : livraisons conformes	4
Jeu caché : livraisons conformes	4
Jeu caché : fichiers mal formés gérés proprement	2
Architecture : perception / mémoire / décision / action séparées	3
Tests	3
Rapport	2
Vidéo de démonstration	1
Git : historique réel, progressif, lisible	1

Un déplacement illégal, une perception falsifiée ou un repli annoncé sans fouille annule les points fonctionnels du jeu concerné.

Soutenance

Vingt minutes par binôme, sans support obligatoire. Le code est ouvert au hasard et les deux membres doivent pouvoir expliquer n'importe quelle portion, y compris celle écrite par l'autre. Une réponse du type « c'est la partie de mon binôme » vaut zéro sur la portion concernée.

11. Le rapport (3 pages maximum)

Quatre points, et rien d'autre :

1. **Architecture** : un schéma, et la justification de vos frontières entre perception, mémoire, décision et action.
 2. **La stratégie de replanification** : que fait votre robot quand il découvre un mur ? Replanifie-t-il tout, ou répare-t-il son plan ? Quel coût ?
 3. **Critique de la règle de repli**. La section 6.2 impose de répondre à une tristesse par l'objet d'une émotion voisine sur la roue — ce qui peut donner, pour une personne triste, un objet rangé sous « dégoût ». Cette règle vous paraît-elle défendable ? Proposez-en une meilleure, et dites ce qu'il faudrait changer dans les fichiers pour la mettre en œuvre.
 4. **Trois échecs de votre robot** : trois situations où il se comporte mal, avec l'analyse de la cause. Un rapport qui affirme que tout fonctionne parfaitement sera lu comme un rapport qui n'a pas cherché.
-

12. Projet 2 — plusieurs robots

Même appartement, mêmes résidents, même armoire, mais **N robots** (N entre 2 et 4) et des demandes qui arrivent **simultanément**. Les règles des sections 3 à 6 restent valables pour chaque robot pris individuellement.

Trois problèmes apparaissent, par difficulté croissante.

12.1 Les collisions

Deux robots ne peuvent pas occuper la même case, ni échanger leurs positions en un tour. Un robot perçoit un autre robot sur une case adjacente comme un obstacle **temporaire** : contrairement à un mur, il ne doit pas être mémorisé comme un obstacle permanent dans la carte mentale.

Votre protocole doit garantir l'absence d'interblocage : deux robots face à face dans un couloir d'une case de large doivent finir par se débloquer. Expliquez pourquoi votre solution termine.

12.2 La répartition des demandes

Quand une demande arrive, quel robot s'en charge ? Vous implémenterez une **enchère** : chaque robot annonce une estimation de son coût (distance estimée sur sa propre carte mentale, plus une pénalité s'il est déjà occupé), le moins-disant emporte la demande.

Notez la difficulté : chaque robot estime son coût sur une carte mentale **différente et incomplète**. Le moins-disant peut se tromper. Que se passe-t-il alors, et que faites-vous ?

12.3 Les ressources partagées

Il n'y a qu'un dictionnaire et qu'une armoire, et un seul sélecteur dans l'armoire. Deux robots ne peuvent pas les utiliser en même temps. Il faut donc un protocole de réservation : définissez-le, et montrez qu'il ne bloque pas.

12.4 Extensions facultatives

- **Partage de carte** : les robots s'échangent les portions de plan qu'ils ont découvertes, lorsqu'ils sont sur des cases adjacentes ou par un canal global — à vous de choisir et de justifier. Mesurez le gain.
- **Gestion du stock** : chaque casier ne contient plus qu'un nombre fini d'objets, décompté à chaque **PRENDRE**. Deux robots qui visent le même casier doivent se départager **avant** de faire le trajet.

Ce qui est évalué en plus

	POINTS
Absence de collision et d'interblocage, démontrée	5
Protocole d'enchère : correction et justification	4
Réservation du dictionnaire et de l'armoire	3
Mesure comparative 1 robot / N robots, sur les mêmes scénarios	4
Réutilisation effective du code du projet 1	2
Rapport	2

La mesure comparative est le cœur du second rendu. Un système multi-agents qui met plus de pas au total qu'un robot unique n'est pas un échec — c'est un résultat, s'il est mesuré, expliqué et discuté.

Annexe A — La roue de Plutchik

#	ÉMOTION	FAIBLE	MOYENNE	FORTE	OPPOSÉE
0	joie	sérénité	joie	extase	tristesse
1	confiance	acceptation	confiance	admiration	dégoût
2	peur	appréhension	peur	terreur	colère
3	surprise	distracted	surprise	stupéfaction	anticipation
4	tristesse	mélancolie	tristesse	chagrin	joie
5	degout	ennui	dégoût	répulsion	confiance
6	colere	contrariété	colère	rage	peur
7	anticipation	intérêt	anticipation	vigilance	surprise

Le sens horaire est celui des indices croissants, modulo 8. C'est aussi l'ordre des colonnes de l'armoire, et la raison pour laquelle elles bouclent.

Annexe B — Se déplacer dans une grille

Cette annexe est là pour ceux à qui l'algorithmique des graphes n'est pas familière. Elle donne tout ce dont vous avez besoin pour le projet.

B.1 Une grille est un graphe

Un **graphe** est un ensemble de points, appelés *sommets*, reliés deux à deux par des *arêtes*. Pour ce projet :

- un **sommet** = une case traversable de l'appartement ;
- une **arête** = le fait que deux cases traversables soient orthogonalement adjacentes.

Vous n'avez rien à construire explicitement : les voisins d'une case `(ligne, colonne)` se calculent à la demande, ce sont `(ligne-1, colonne)`, `(ligne+1, colonne)`, `(ligne, colonne-1)` et `(ligne, colonne+1)`, filtrés par « est-ce traversable ».

Le problème « aller de A à B en un minimum de pas » est donc le problème du **plus court chemin dans un graphe où toutes les arêtes coûtent 1**. Ce cas particulier a une solution simple, le **parcours en largeur**.

B.2 Le parcours en largeur

L'idée : explorer les cases par distance croissante depuis le départ. D'abord toutes les cases à 1 pas, puis toutes celles à 2 pas, et ainsi de suite. La première fois qu'on atteint la cible, on y est forcément arrivé par un plus court chemin, puisqu'on a épuisé toutes les distances inférieures avant.

Pour respecter cet ordre, on utilise une **file** : on ajoute les cases découvertes à la fin, et on traite toujours celle qui attend depuis le plus longtemps (premier arrivé, premier servi).

C'est le seul point vraiment important. Si vous utilisez une **pile** (dernier arrivé, premier servi), vous obtenez un parcours en profondeur : il trouve *un* chemin, mais pas le plus court.

B.3 Un exemple déroulé

Prenons cette mini-grille, avec le robot en `R = (1, 1)` et une cible en `(1, 4)` :

```
      colonne 0 1 2 3 4 5
ligne 0      # # # # # #
ligne 1      # R . # . #
ligne 2      # . # . . #
ligne 3      # . . . # #
ligne 4      # # # # # #
```

Le parcours en largeur numérote les cases par distance au départ :

```
      colonne 0 1 2 3 4 5
ligne 0      # # # # # #
ligne 1      # 0 1 # 7 #
ligne 2      # 1 # 5 6 #
ligne 3      # 2 3 4 # #
ligne 4      # # # # # #
```

La case (1, 2), voisine immédiate du départ, est à 1 pas — mais c'est un cul-de-sac : le mur en (1, 3) bloque le passage direct. Le chemin réel descend, contourne par le bas, et remonte :

(1,1) → (2,1) → (3,1) → (3,2) → (3,3) → (2,3) → (2,4) → (1,4), soit 7 pas.

Un algorithme naïf qui irait « toujours vers la cible » se serait coincé en (1, 2). Le parcours en largeur, lui, explore tout à distance 1, puis tout à distance 2, et trouve le contournement sans effort.

B.4 Le pseudo-code

```
fonction plus_court_chemin(depart, arrivees, est_traversable) :
    si depart ∈ arrivees :
        renvoyer [depart]

    predecesseur ← table associative vide
    predecesseur[depart] ← rien
    file ← file vide
    enfiler(file, depart)

    tant que file n'est pas vide :
        courant ← defiler(file)          # le plus ancien !
        pour chaque voisin de courant :
            si voisin est déjà dans predecesseur : passer au suivant
            si non est_traversable(voisin)      : passer au suivant
            predecesseur[voisin] ← courant
            si voisin ∈ arrivees :
                renvoyer reconstruire(predecesseur, voisin)
            enfiler(file, voisin)

    renvoyer « aucun chemin »
```

La table `predecesseur` sert deux fois : elle marque les cases déjà vues, ce qui évite de tourner en rond, et elle mémorise par où on est arrivé, ce qui permet de reconstruire le chemin à l'envers :

```
fonction reconstruire(predecesseur, case) :
    chemin ← [case]
    tant que predecesseur[dernier élément de chemin] ≠ rien :
        ajouter predecesseur[dernier élément de chemin] à chemin
    renvoyer chemin inversé
```

Le coût est proportionnel au nombre de cases : chaque case entre au plus une fois dans la file. Sur un appartement de 15 × 21, c'est instantané, même recalculé à chaque pas.

Piège d'implémentation en Python. N'utilisez pas une liste avec `file.pop(0)` : retirer le premier élément d'une liste coûte cher et votre programme ralentira inutilement. Utilisez `collections.deque` avec `append` et `popleft`.

B.5 Planifier sur une carte incomplète

Votre robot ne connaît pas les murs. La solution tient en une phrase : **il suppose libre tout ce qu'il n'a pas encore vu**. C'est ce qu'on appelle une hypothèse optimiste.

La boucle devient :

1. percevoir les quatre cases voisines, et les inscrire dans la carte mentale ;

2. calculer un plus court chemin sur la carte mentale ;
3. faire **un seul pas** de ce chemin ;
4. recommencer.

Il n'est pas nécessaire de détecter que le plan est devenu faux : comme on replanifie à chaque pas sur une carte qui vient d'être mise à jour, le chemin se corrige tout seul dès que le mur est découvert.

Cette approche recalcule beaucoup, et c'est exactement le compromis à discuter dans votre rapport (section 11, point 2).

B.6 Pourquoi l'échec du parcours est une preuve

La carte mentale contient **au moins autant de cases libres** que la réalité : les cases vues sont exactes, et les cases inconnues sont supposées libres alors qu'elles sont peut-être des murs.

Donc, si un chemin existe dans la réalité, il existe aussi dans la carte mentale. Par contraposée : **si le parcours en largeur ne trouve aucun chemin sur la carte mentale, c'est qu'il n'en existe aucun dans la réalité.**

C'est ce qui permet de conclure proprement qu'un résident est inaccessible, sans exploration exhaustive de l'appartement : il suffit d'explorer jusqu'à ce que le parcours échoue. Et cela finit toujours par arriver, puisque chaque déplacement révèle des murs et ne peut donc que réduire le nombre de cases supposées libres.

B.7 Et dans l'armoire ?

Le sélecteur se déplace lui aussi sur une grille, mais elle est minuscule et sans obstacle : pas besoin de parcours en largeur, la distance se calcule directement.

Pour aller du casier (l, c) au casier (l', c') :

- verticalement, $|l' - l|$ mouvements N ou S ;
- horizontalement, $\min(a, 8 - a)$ mouvements, où $a = (c' - c) \bmod 8$ — on prend le sens le plus court, puisque les colonnes bouclent. Si $a \leq 8 - a$, on va vers l'E, sinon vers l'O.

Annexe C — Naviguer dans l'armoire

L'armoire est une grille minuscule, mais elle a deux particularités qui surprennent : ses colonnes bouclent, et l'itinéraire du sélecteur n'est pas imposé. Cette annexe lève les deux ambiguïtés.

C.1 Le repérage

Le casier $[ligne, colonne]$ se lit ainsi :

	0 JOIE	1 CONFIANCE	2 PEUR	3 SURPRISE	4 TRISTESSE	5 DÉGOÛT	6 COLÈRE	7 ANTICIPATION
0 faible	[0,0]	[0,1]	[0,2]	[0,3]	[0,4]	[0,5]	[0,6]	[0,7]
1 moyenne	[1,0]	[1,1]	[1,2]	[1,3]	[1,4]	[1,5]	[1,6]	[1,7]
2 forte	[2,0]	[2,1]	[2,2]	[2,3]	[2,4]	[2,5]	[2,6]	[2,7]

Le fichier d'armoire donne, pour chaque casier, sa **ligne**, sa **colonne**, son **emotion** et son **intensite** : les quatre sont cohérents entre eux, et vous avez le droit de les lire. Seul le champ **objet** vous est interdit.

C.2 Les mouvements

CHERCHER N et CHERCHER S changent la ligne, CHERCHER E et CHERCHER O changent la colonne.

- N depuis la ligne 0 et S depuis la ligne 2 sont **illégaux** : les lignes ne bouclent pas.
- E depuis la colonne 7 mène à la colonne 0, et O depuis la colonne 0 mène à la colonne 7 : les colonnes, elles, **bouclent**.

C.3 Compter les mouvements

Pour aller du casier $[l, c]$ au casier $[l', c']$:

- verticalement**, $|l' - l|$ mouvements, S si $l' > l$, N sinon ;
- horizontalement**, on pose $a = (c' - c) \bmod 8$. Si $a \leq 8 - a$, ce sont a mouvements E ; sinon ce sont $8 - a$ mouvements O.

Trois exemples :

DE	VERS	VERTICAL	HORIZONTAL	TOTAL
[1,0]	[1,4]	aucun	$a = 4$, $8-a = 4$, donc $4 \times E$	4
[2,7]	[0,1]	$2 \times N$	$a = 2$, donc $2 \times E$	4
[1,0]	[2,6]	$1 \times S$	$a = 6$, $8-a = 2$, donc $2 \times O$	3

Le troisième est celui qui piège : pour aller de la colonne 0 à la colonne 6, il est plus court de reculer de deux crans que d'avancer de six. C'est tout l'intérêt du bouclage.

C.4 L'ordre des mouvements est libre

Reprenons $[1,0] \rightarrow [2,6]$, qui coûte trois mouvements. Deux itinéraires possibles :

vertical d'abord	S	O	O	horizontal d'abord	O	O	S
	[2,0]	[2,7]	[2,6]		[1,7]	[1,6]	[2,6]

Les deux sont **acceptés**, et coûtent la même chose. Mais ils ne font pas passer devant les mêmes casiers : le premier montre $[2,0]$ et $[2,7]$, le second $[1,7]$ et $[1,6]$. Comme le robot lit le contenu de tout casier devant lequel son sélecteur se trouve, le choix de l'itinéraire décide de ce qu'il apprend en chemin.

Vous n'êtes d'ailleurs pas obligés de prendre le chemin le plus court. Un robot qui balaie toute l'armoire à chaque demande produit une trace parfaitement valide — simplement très longue.

C.5 Une fouille déroulée

Demande de **surprise / forte**, avec l'armoire standard dont toute la colonne **surprise** est vide. La règle de la section 6.2 donne cet ordre de candidats :

RANG	CASIER	ÉMOTION / INTENSITÉ	REPLI	CONTENU
1	[2,3]	surprise / forte	aucun	vide
2	[1,3]	surprise / moyenne	intensite	vide
3	[0,3]	surprise / faible	intensite	vide
4	[2,4]	tristesse / forte	voisine_1	téléphone
5	[2,2]	peur / forte	voisine_1	bouton d'appel

Le sélecteur part de [1,0] . Une fouille correcte donne :

```
[1,0] --S--> [2,0] --E--> [2,1] --E--> [2,2] --E--> [2,3]   vide, rang 1
      --N--> [1,3]                                     vide, rang 2
      --N--> [0,3]                                     vide, rang 3
      --S--> [1,3] --S--> [2,3] --E--> [2,4]             telephone, rang 4
PRENDRE telephone      casier_choisi = [2, 4]   repli = voisine_1
```

Neuf mouvements, puis **PRENDRE** .

Deux choses à remarquer.

D'abord, le robot est passé devant [2,2] — peur/forte , qui est garni — dès son quatrième mouvement. Il ne le prend pas : peur/forte est le rang 5, et tristesse/forte le rang 4. **L'ordre prime sur ce qu'on croise.** Les deux sont pourtant à la même distance sur la roue et à la même intensité ; c'est le critère du sens horaire qui les départage.

Ensuite, il repasse par [1,3] et [2,3] , déjà visités et déjà connus vides. Ce n'est pas une faute, c'est le prix de l'aller-retour. Un robot plus malin descendrait la colonne **surprise** dans l'autre sens, ou anticiperait la suite de la liste. C'est un bon sujet pour la partie « échecs » de votre rapport.

Annexe D — Une simulation complète

Carte **appartement_01** , robot en [6, 1] , armoire en [6, 4] , dictionnaire en [7, 6] , R1 « Elodie » en [6, 11] , R2 « Daniel » en [3, 3] .

```

      colonne 0123456789012
ligne 0 #####
ligne 1 #.....#....#
ligne 2 #.....#....#
ligne 3 #..P...#....#
ligne 4 #.#####.#
ligne 5 #.....#....#
ligne 6 #R..A..#...P#
ligne 7 #.....D#....#
ligne 8 #####

```

Demande 1 — R1, « Je me sens tellement seule ce soir, personne n'est passé. »

1. Le robot rejoint une case adjacente au dictionnaire [7, 6] — par exemple [7, 5] — et exécute **CONSULTER**. Premier mot reconnu : **seule** → **tristesse / moyenne**.
2. Il rejoint une case adjacente à l'armoire [6, 4], par exemple [6, 5]. Le premier candidat est le casier exact **tristesse/moyenne**, en [1, 4]. Le sélecteur part de [1, 0] : quatre mouvements **E** suffisent. Le casier contient « couverture » : **PRENDRE couverture**, **repli** = **aucun**.
3. Il rejoint une case adjacente à R1 en [6, 11] et exécute **DONNER couverture**.

Deux remarques sur cette carte, qui est la plus facile du lot.

D'abord, la partie droite de l'appartement n'est reliée au reste que par la case [4, 11]. Un robot qui part vers l'est en ligne 6 se heurte au mur [6, 7], qu'il ne découvre qu'en arrivant en [6, 6], et doit alors replanifier en remontant par la ligne 2. C'est exactement le comportement attendu.

Ensuite, l'armoire et le dictionnaire sont proches l'un de l'autre. Le trajet « dictionnaire puis armoire » coûte donc peu, alors que le retour vers R1 est long. Selon l'ordre dans lequel votre parcours départage les chemins de même longueur, votre robot passera par des cases différentes des nôtres : les deux sont corrects, et c'est le genre de choix à expliquer dans le rapport.